Secure File Upload System

Final Project Report

Student: [Student name]

Enrollment number: [Enrollment number]

Programme and semester: [Programme and semester]

Department: [Department]

Institution: [College or university]

Guide: [Guide name and designation]

Academic year: [Academic year]

Academic project documentation

Contents

Submission details

Abstract

1 Introduction

2 Research and feasibility

3 Software requirements specification

4 Secure SDLC

5 Architecture and data flow

6 Database design

7 Implementation and security controls

8 User interface and experience

9 Testing and results

10 Deployment and operation

11 Risks and limitations

12 Future enhancement plan

13 Conclusion

References

Submission details

This report documents the design, implementation and evaluation of the Secure File Upload System. The accompanying source code provides the implementation evidence. The synopsis gives the proposal-level overview; the security, deployment, testing and research documents contain operational details. Institutional declarations, certificates and signatures must be completed only by the student and authorized signatories.

Declaration field: [Insert the institution-approved declaration after reviewing authorship and assistance-disclosure requirements.]

Certificate field: [Insert the college-approved certificate and obtain the required signatures.]

Acknowledgements field: [Add the names and contributions of people who actually assisted the project.]

Abstract

The Secure File Upload System provides an authenticated web workflow for accepting a restricted set of text and image files. Its design addresses upload-related threats through independent validation layers, image normalization, access control, resource limits and authenticated encryption. A file is not accepted solely because its extension or declared MIME type appears valid. PNG and JPEG uploads must also match their signatures and decode successfully; text uploads must satisfy UTF-8 and control-character rules.

The implementation combines Flask, SQLite, Pillow, cryptography, PyOTP and Waitress. Security features include owner-bound downloads, CSRF protection, scrypt password hashing, TOTP replay prevention, revocable sessions, account quotas and a per-user HMAC audit chain. A configurable ClamAV adapter provides a fail-closed scanning path when a scanner is available. The project is evaluated through automated tests and local browser inspection. This report records the complete SDLC, requirements, architecture, data model, implementation choices, test strategy, operational procedures and known limitations. It does not claim that automated tests establish complete security or that an unconfigured antivirus adapter performs malware detection.

1 Introduction

1 1 Background

File upload is a trust-boundary crossing: a remote user supplies bytes that the server may parse, store and later distribute. The server must determine which operations are permitted and which assumptions about those bytes are justified. Unrestricted storage can expose application code execution, resource exhaustion, information disclosure and unauthorized retrieval paths. A private upload application must therefore control both ingestion and later access.

This project focuses on files that can be processed in a small, explainable pipeline. TXT, PNG, JPG and JPEG are accepted. PDF, Office documents, SVG, archives and executable formats are excluded because they introduce different parser, active-content and decompression concerns that are not handled by the implemented pipeline.

1 2 Problem definition

The problem is to build an upload system that rejects inappropriate inputs, preserves the confidentiality of stored content and enforces ownership throughout the file life cycle. The system must remain usable for a student demonstration and its claims must correspond to verifiable behavior. Filename checks, encryption and antivirus are separate controls; none can substitute for the others.

1 3 Objectives and success criteria

Success means a new user can register, sign in, upload a supported file, see it in a private list, download the verified normalized content and delete it. A second user must not obtain that file by copying its identifier. Invalid files must create neither accepted metadata nor retained encrypted blobs. Account protection and audit features must work through the interface and have meaningful regression tests.

The secondary objective is an academic submission package that explains why each control exists, how it is implemented and what remains outside scope. The project should support a viva in which the student can distinguish MIME declarations, file signatures, parsers, normalization, encryption, integrity and authorization.

2 Research and feasibility

2 1 Reference review

OWASP's upload guidance informs the layered ingestion design [R1]. Its password-storage guidance informs the explicit scrypt configuration rather than a fast unsalted digest [R2]. NIST SP 800-218 supplies the secure-development framework used to connect planning, implementation, verification and maintenance activities [R3]. The Flask security documentation informs browser and application safeguards [R6]. Official cryptography, Pillow, PyOTP and ClamAV documentation are used for library behavior [R4, R5, R7, R8].

This is an engineering reference review, not a systematic literature review or a statistical comparison of commercial products. The application architecture, policy values and acceptance tests are project-specific design decisions. The research register contains direct links, implementation connections and extension ideas.

2 2 Baseline comparison

Dimension	Minimal upload example	This project
Validation	Trust extension or browser MIME	Independent name, MIME, signature and content checks
Privacy	Preserve uploaded bytes and metadata	Normalize images; encrypt stored content
Retrieval	Public directory URL	Authenticated owner route with integrity checks
Account security	Password and browser cookie	scrypt, optional TOTP, server revocation
Resource management	Unbounded or request-only limit	Request, file, pixel, byte-quota and count limits
Evidence	Manual success demonstration	Automated negative tests and audit history

2 3 Feasibility

Technical feasibility comes from a monolithic application with limited dependencies and a supported embedded database. SQLite avoids a separate database service for the demonstration, while WAL mode and bounded transactions support small concurrent workloads. Economic feasibility comes from local execution and open-source libraries; optional hosting and antivirus operations introduce separate resource costs. Operational feasibility is supported by a single account role and direct upload/download controls.

Schedule feasibility depends on constraining supported formats and avoiding unfinished features disguised as implemented modules. The implemented version may be reviewed incrementally, while public deployment remains a separate environment-specific acceptance step. Suggested development resources are a 64-bit machine with 4 GB RAM and 1 GB free disk; these are planning estimates and should not be described as benchmarked requirements.

3 Software requirements specification

3 1 Stakeholders and actors

The student developer implements and explains the system. The project guide reviews scope and evidence. A registered user manages only their own files and security settings. An operator configures the server, keys, backups and optional scanner through deployment access. The operator is not an application administrator role and no administrator dashboard is implemented. ClamAV is an optional external process invoked by the server.

3 2 Functional requirements

ID	Requirement	Implementation evidence
FR01	Validate registration and store a password hash	Register route and users table
FR02	Authenticate credentials and optionally TOTP	Login route and last accepted TOTP step
FR03	Maintain expiring, revocable sessions	Sessions table and per-request lookup
FR04	Apply filename, MIME and content policies	Upload validation function
FR05	Normalize supported images	Pillow verification, decode and re-encode
FR06	Bound request and account resource use	Request limits and transactional quota check
FR07	Encrypt accepted content under random IDs	Fernet and private vault directory
FR08	Authorize and verify each download	Owner query, decryption and SHA-256 comparison
FR09	Delete an owned file and its metadata	POST delete route
FR10	Show personal activity with integrity status	Activity route and HMAC verification
FR11	Configure and enforce optional antivirus	Scanner adapter and required mode
FR12	Allow MFA setup and all-device sign-out	Security page and revocation route

3 3 Nonfunctional requirements

ID	Quality attribute	Acceptance approach
NFR01	Confidentiality	Cross-account denial and ciphertext-at-rest tests
NFR02	Integrity	Corrupted ciphertext and altered-audit tests
NFR03	Availability bounds	Size, pixel, quota, timeout and rate-limit tests
NFR04	Usability	Labeled forms, meaningful errors and browser walkthrough
NFR05	Maintainability	App factory, pinned direct dependencies and regression tests
NFR06	Persistence	Restart test using the same data and key directory
NFR07	Portability	Python entry point plus container assets; validate each target separately
NFR08	Evidence accuracy	Separate measured results, mocked integrations and future work

3 4 Business rules

Usernames are normalized to lowercase and contain 3-32 ASCII letters, digits or underscores. Passwords contain 12-128 characters. A file basename begins with an alphanumeric character, has a bounded stem containing letters, digits, spaces, underscores or hyphens, and ends with one allowed extension. A file is nonempty and at most 5 MiB; a decoded image is at most 16 megapixels. Each user may store at most 50 MiB of normalized plaintext and 200 files. Encryption and database overhead do not count toward the displayed quota.

An account's session lasts at most 30 minutes from sign-in. Files are never publicly shareable. Images are normalized before scanning and storage; downloaded image bytes may differ from the original. Security controls execute on the server regardless of client JavaScript. If a configured scanner fails, the file is not stored. Missing scanner configuration is allowed in disclosed demonstration mode and denied in required-scanning mode.

3 5 Use cases and misuse cases

UC01 Register: the visitor submits a valid username and password; the server creates a user and redirects to sign-in. Duplicate names and invalid lengths produce explanatory errors.

UC02 Upload: an authenticated user submits a file and CSRF token; the server validates, normalizes, optionally scans, enforces quotas, encrypts and records the accepted content. A policy failure returns an error and a rejection event.

UC03 Download: a user requests a file identifier; the server verifies ownership and integrity, then sends an attachment. Missing ownership returns 404. Corrupt or missing stored content returns 409.

UC04 Protect account: the user confirms their password, scans a locally generated QR code, proves possession with a TOTP code and enables MFA. They can later disable it with a password and fresh code, or revoke all sessions.

Misuse cases include an attacker supplying a path traversal name, a PHP file with an image suffix, a forged MIME declaration, an image decompression bomb, another user's file ID, a missing CSRF token, a reused TOTP code or modified encrypted content. These cases guide negative testing rather than merely documenting happy paths.

4 Secure SDLC

4 1 Development model

An iterative secure SDLC is used. Requirements and threats define the first increment; implementation and tests then refine the design. The model permits correcting a control when evidence reveals a gap. Security is addressed in each phase rather than postponed to a final test. The project uses NIST SSDF as a reference for organizing practices and does not claim formal NIST certification [R3].

4 2 Planning and requirements

Planning establishes the project problem, submission outputs, technical constraints and accepted formats. Requirements identify assets such as passwords, file bytes, metadata, encryption keys and activity history. Misuse cases become acceptance tests. Exit criteria are a coherent synopsis, an SRS with observable outcomes and an explicit out-of-scope list.

4 3 Analysis and design

Design separates the browser, application, data store, encrypted filesystem and scanner process. The threat model assumes malicious clients but a trusted operating system and operator. It defines ownership checks, key boundaries, request limits and failure behavior. Exit criteria are an architecture diagram, schema, processing sequence and reviewed security tradeoffs.

4 4 Implementation

Implementation proceeds through account access, core file operations, content checks, cryptography, MFA and audit integrity. User input enters parameterized SQL. Escaped templates avoid embedding untrusted strings as executable markup. Supported libraries implement cryptographic and image primitives. Direct dependency versions and configuration examples make the application reproducible.

4 5 Verification and validation

Verification tests whether modules meet their specified behavior. Validation checks that a student can complete the intended workflow and demonstrate the controls. Negative tests cover rejection paths, cross-account access and tampering. Browser inspection checks the actual interface. A test failure is corrected and the relevant suite is rerun. Runtime-specific limitations are recorded separately from code defects.

4 6 Deployment and maintenance

Deployment configures persistent storage, process startup, keys, scanner policy and transport security. Local deployment binds to loopback. Remote deployment requires HTTPS and environment-specific verification. Maintenance includes dependency review, signature updates, backup checks, incident response and regression tests after security-relevant changes. The GitHub workflow automates tests and code/dependency scans when the source is published to a repository with Actions enabled.

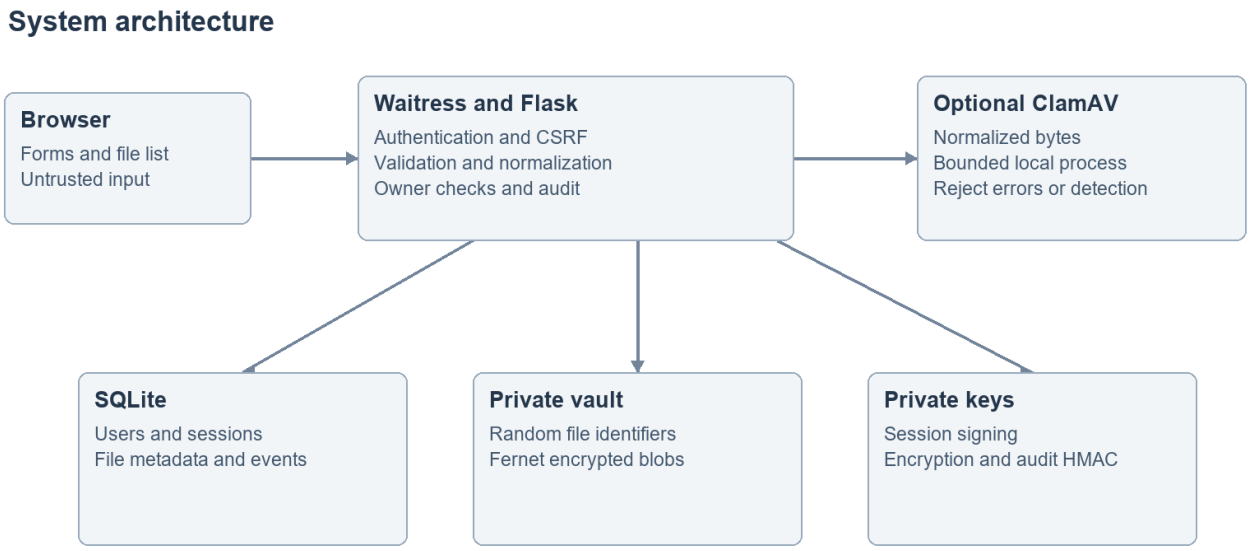
4 7 Requirements traceability

Requirement	Design component	Test or review evidence
FR01-FR03	Identity and session module	Password hash, authentication, expiry and copied-cookie tests
FR04-FR05	Validation and normalization	MIME mismatch, magic/structure, EXIF and dimension tests
FR06	Resource-control layer	File/request size, account quota and count tests
FR07-FR09	Private vault module	Encryption/download/delete and cross-user tests
FR10	Audit chain	Event alteration detection and user-isolation tests
FR11	ClamAV adapter	Controlled success, rejection, error and timeout tests
FR12	Security settings	Enrollment, replay prevention and all-device logout tests
NFR04	Interface	Browser login, dashboard, settings and small-screen inspection
NFR06	Persistent keys and database	Restart with existing encrypted file

5 Architecture and data flow

5 1 Application architecture

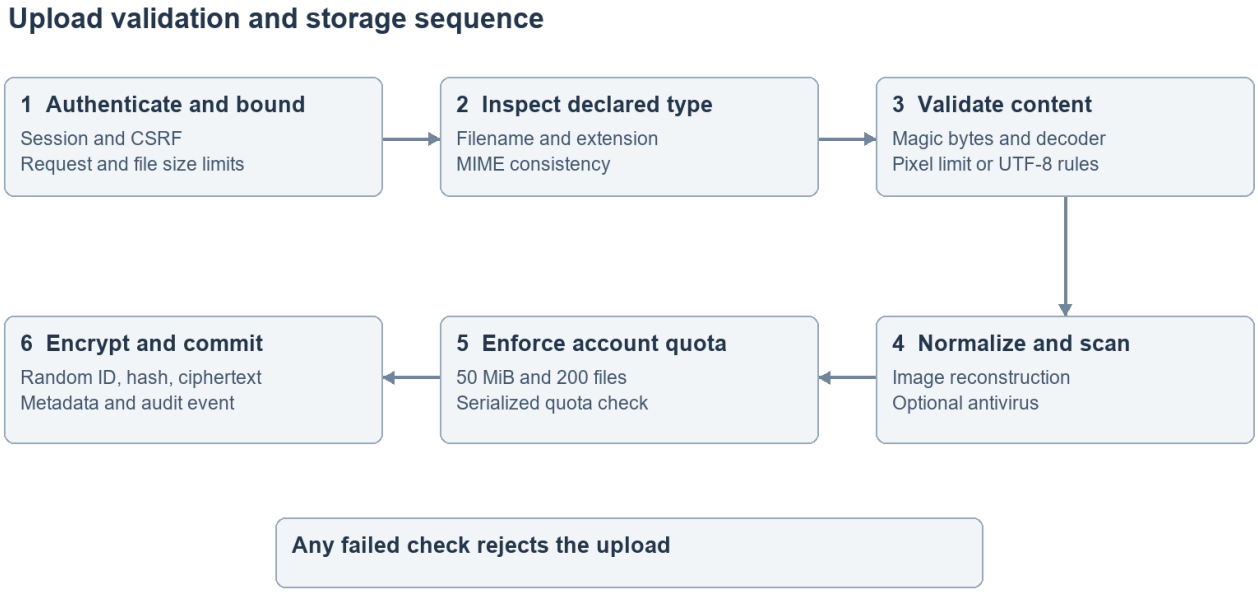
The system is a small server-rendered web application. Waitress receives HTTP requests and invokes the Flask application. Jinja renders HTML with escaped values; JavaScript provides file-size feedback and deletion confirmation without determining authorization. The database stores identity, sessions, metadata and events. The vault directory stores encrypted blobs. Three key files serve session signing, content encryption and audit authentication.



System architecture

5 2 Data flow and trust boundaries

The browser-to-server boundary is untrusted input. The optional scanner is a separate process with a bounded execution time. Uploaded file names are display metadata and never become filesystem paths. Data moves into trusted storage only after the processing pipeline accepts it. The download route performs a new ownership check; an unguessable identifier is not treated as authorization.



Upload processing flow

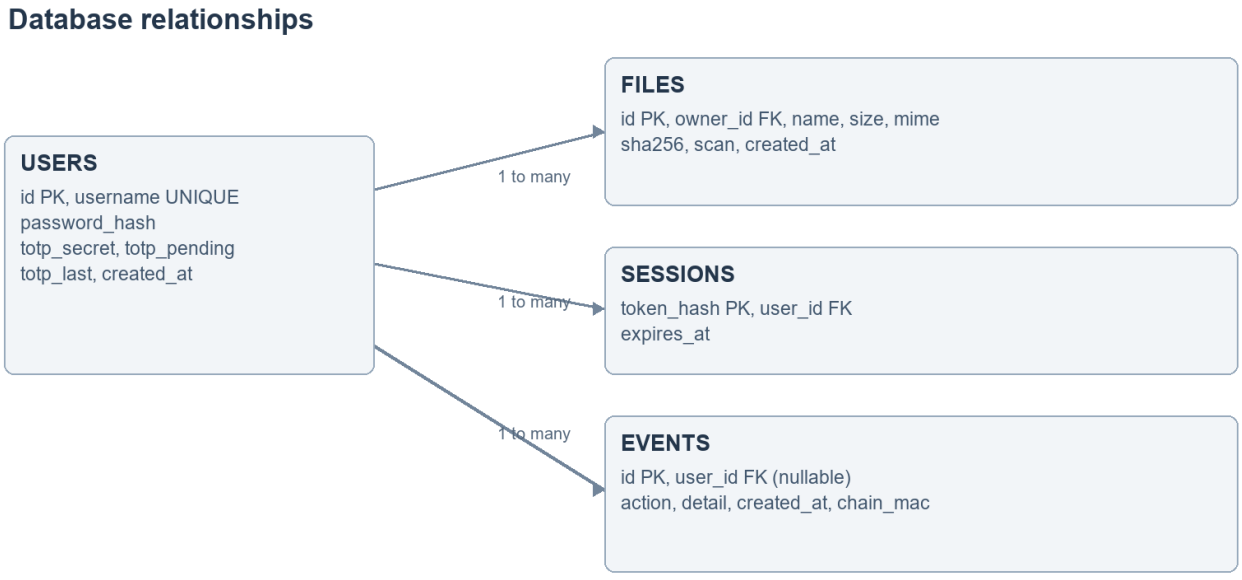
5 3 Main components

Component	Responsibility
Account routes	Registration, sign-in and current-user lookup
Security routes	TOTP enrollment, confirmation, disable and session revocation
Validation function	Filename, MIME, byte, text and image checks
Scanner adapter	Temporary normalized file and controlled scanner process
Storage routes	Transactional quota, encryption, metadata and authorized retrieval
Audit functions	HMAC-linked events and retained-chain verification
Templates and styles	Accessible forms, file listing, activity and account settings
Test suite	Isolated application instances and negative-path assertions

6 Database design

6 1 Entity relationships

One user owns many files, has many session records and has many activity events. Events for unknown authentication attempts may have a null user reference. File identifiers are random hexadecimal strings. The schema uses foreign keys and indexes for owner-based queries. The relationship diagram and complete Mermaid source are included in the diagrams document.



Database relationships

6 2 Data dictionary

Table	Important fields	Meaning
users	id, username, password_hash, created_at	Account identity and salted password representation
users	totp_secret, totp_pending, totp_last	Encrypted MFA secrets and highest consumed time step
sessions	token_hash, user_id, expires_at	Hashed random session token and absolute Unix expiry
files	id, owner_id, name, size, mime	Internal file key, owner and normalized-content metadata
files	sha256, scan, created_at	Fingerprint, scanner status and upload time
events	id, user_id, action, detail, created_at	Audit event content
events	chain_mac	HMAC linking the account's event sequence

6 3 Consistency and transactions

The application opens one SQLite connection per request context and closes it afterwards. Upload quota checking and metadata insertion execute under an immediate write transaction to prevent simultaneous requests from both consuming the same remaining quota. File and database operations cannot share a native transaction; ordinary upload failures roll back metadata and remove the new blob. A process crash between filesystem and database operations can leave an orphan, and deletion can leave missing content if interrupted. Backup/restore procedures and future reconciliation tooling address this operational boundary.

Schema initialization uses idempotent table creation and narrowly scoped column additions for MFA and audit fields. A full migration framework is future work. Large deployments should evaluate a server database and explicit migration tooling rather than extending ad hoc initialization indefinitely.

7 Implementation and security controls

7 1 MIME and magic-byte validation

The file extension defines the permitted processing path. A specific declared MIME must agree with it; a missing or generic binary declaration still requires full content checks. PNG requires the eight-byte PNG signature and JPEG requires the leading FF D8 FF bytes. Pillow then identifies the format independently, verifies structure and decodes the pixels. A magic signature is a preliminary consistency check, not proof of harmless content.

TXT has no unique binary signature. The implementation requires valid UTF-8 and excludes non-text C0 controls except tab, carriage return and newline. A text file may contain code-like text; attachment delivery prevents treating it as a web page, but users must still avoid executing untrusted downloads.

7 2 Normalization and metadata removal

Images are decoded under a 16-megapixel ceiling, converted to RGB for JPEG or RGBA for PNG, stripped of metadata and encoded again. Only the output is scanned, hashed, encrypted and retained. This removes EXIF/GPS metadata and appended bytes from the supported image pipeline. JPEG output can be lossy and color-profile removal may change appearance. Animated input is flattened to the first decoded frame. Those transformations must be explained to users rather than described as byte-preserving storage.

7 3 Encryption and integrity

Fernet supplies authenticated encryption through a maintained high-level recipe. It uses AES-128-CBC with HMAC-SHA-256 and a random initialization vector; the project does not implement AES itself [R4]. SHA-256 fingerprints normalized plaintext so the stored digest and downloaded content can be compared. Decryption or digest failure stops delivery and records an event.

The server can decrypt because it holds the key. Encryption at rest does not protect against a fully compromised server, and it is not end-to-end encryption. Key loss makes both file content and encrypted TOTP secrets unrecoverable. The session-signing and audit keys are independent of the content-encryption key.

7 4 Authentication and sessions

Passwords use salted scrypt with explicit N=32768, r=8, p=3 parameters. A dummy password verification reduces one source of username timing differences. Login attempts are rate limited. A successful login creates a random token whose hash is stored in SQLite. Signed cookies use HttpOnly and SameSite=Lax, and Secure is enabled for HTTPS deployment. Server-side expiry and revocation prevent a copied cookie from remaining valid after logout.

7 5 Two-factor authentication

The security page requires a current password before beginning enrollment. The server generates an encrypted pending secret and a local QR image. Confirmation with a valid code enables MFA and revokes other sessions. A database update consumes each time step only once, preventing immediate code replay. MFA disable and all-device sign-out require fresh verification when MFA is enabled. Account recovery and phishing-resistant WebAuthn are outside the implemented release.

7 6 Audit trail

Events include registrations, sign-ins, file actions, rejected uploads, integrity failures and security-setting changes. Each account has a separate HMAC chain so modification of retained events can be detected. The activity screen checks the chain and displays the latest 100 events. The audit key must be protected separately from the database. Without an external checkpoint, the chain does not detect removal of a final suffix; it is tamper-evident rather than immutable.

7 7 Antivirus integration

When configured, the application writes normalized bytes to a temporary private file and invokes a fixed scanner executable with an argument array, never a shell-composed command. A clean scanner result permits further processing. Detection, operational errors and timeout reject the upload. Demonstration mode explicitly displays that files were not scanned when ClamAV is absent. Tests of the adapter simulate return codes and do not establish the detection quality of a live scanner [R8].

8 User interface and experience

The interface emphasizes files and account operations. The sign-in screen identifies the application and supported workflow without claiming that uploaded content is guaranteed safe. The dashboard shows a file list, byte usage and upload policy. Errors state which rule failed. Download and delete actions appear beside each file, while SHA-256 details remain expandable. Activity and security settings are distinct pages.

HTML labels, keyboard-focus outlines and status messages support basic accessibility. Responsive layouts move panels into a single column on small screens. Client JavaScript improves feedback but cannot authorize uploads or bypass server validation. Delete confirmation is a convenience; CSRF and ownership checks provide the actual server-side protection. Visual inspection is part of validation and is not represented as a formal WCAG audit.

9 Testing and results

9 1 Test strategy

Tests run against temporary application instances, independent databases and generated keys. Functional tests cover the intended workflow. Security tests cover boundaries such as forged MIME, misleading signatures, cross-user requests, request limits, missing CSRF, corrupted storage and replayed authentication material. Persistence tests reopen the same data directory. Controlled scanner tests verify integration behavior without requiring live malware samples.

9 2 Test groups

Group	Representative checks	Expected result
Identity	Valid account, scrypt representation, anonymous requests	Correct login; anonymous access redirected
Input validation	PHP, SVG, ZIP, PDF, unsafe names, fake images, invalid UTF-8	No accepted file record
Content processing	Valid image, EXIF marker, appended marker, excessive pixels	Safe normalization or rejection
Isolation	Other user's list, activity, download and delete	No content exposure or unauthorized deletion
Integrity	Ciphertext corruption and event modification	Download blocked or audit warning
Resource bounds	File/request size, byte quota, file count, login throttling	Rejection at documented boundaries
Sessions and MFA	Expiry, logout replay, all-device revoke, TOTP replay	Invalidated authentication rejected
Scanner	Clean, infected, error, timeout, absent-required	Correct accept/reject; temporary cleanup

9 3 Execution evidence and interpretation

The current machine-readable execution evidence is docs/test-results.xml; the human-readable run record is docs/TESTING.md. Those files record actual test outcomes, environment details and unverified integrations. Tests are rerun after security-relevant changes. A passing suite demonstrates the asserted cases for that version; it does not establish protection against every parser vulnerability, distributed denial-of-service attack or operational misconfiguration.

9 4 Defect handling

Each failure is classified as application behavior, test expectation or environment limitation. The development environment required an unrestricted local test run because temporary-directory access was blocked by its execution sandbox. A disabled rate-limiter lifetime issue was corrected by retaining the limiter on the application. Such issues are resolved through observable tests rather than hidden behind a blanket statement that all functionality works.

10 Deployment and operation

10 1 Local deployment

Create a Python virtual environment, install the pinned requirements and start run.py. The Waitress process binds to 127.0.0.1:5000 by default. Create an account through the browser; no default credentials are provided. The application creates the private data directory and keys on first startup. Preserve that directory between runs.

10 2 Container and remote deployment

The Dockerfile runs as a non-root user. Compose binds the published port to loopback, uses a persistent volume, drops capabilities and applies a read-only root filesystem. Container assets are supplied separately from their runtime verification status. A remote deployment requires a TLS-terminating proxy, secure cookies, protected data permissions, a scanner policy, rate-limit backend decisions and backup validation. A Docker configuration alone does not mean the application has been deployed publicly.

10 3 Backup and restore

For the academic deployment, stop the application before copying the complete data directory so database, blobs and keys form a consistent snapshot. Protect backups with equivalent or stronger access controls and encryption. Restore to a separate private directory, point DATA_DIR at it and verify a known file download. Preserve session, encryption and audit keys with the database. Loss or substitution of keys is not repaired by recreating empty key files.

10 4 Monitoring and maintenance

Monitor disk use, repeated rejection events, authentication failures and integrity warnings. Review dependencies and scanner signatures regularly. Keep operating-system and container images current. Review GitHub Actions findings before promoting a new version. A future operator interface may centralize alerts, but the implemented interface exposes only a user's personal activity.

11 Risks and limitations

The main residual risk is compromise of the trusted server or its keys. Other limitations are the narrow accepted-format set, no account recovery, no live antivirus by default, in-memory rate limiting, no global signup or server-disk quota, and limited crash reconciliation. Database audit integrity requires a protected key and independent checkpoints to detect truncation. The application has not undergone an independent penetration test or formal compliance review.

MIME and magic checks are necessary consistency tests but not complete malware detection. Image re-encoding does not eliminate every possible decoder vulnerability. Encryption does not prevent users from uploading sensitive material or executing a dangerous file after download. These limitations define the demonstration boundary and motivate the extension roadmap.

12 Future enhancement plan

Priority	Enhancement	Required design and validation
1	WebAuthn passkeys and recovery codes	Origin-bound credentials, enrollment/recovery threats and replay tests
1	External audit checkpoints	Independent storage of chain heads and truncation tests
1	Key manager and rotation	Versioned envelope encryption, access policy and recovery drills
2	Quarantined asynchronous scanning	Queue state machine, scanner sandbox, retries and fail-closed downloads
2	PDF or Office support	Format-specific parsing, CDR, active-content policy and adversarial corpus
2	Shared rate limiter and PostgreSQL	Multi-instance concurrency tests and migrations
3	Cloud object storage	Private buckets, scoped access, encryption and tenant-isolation tests
3	Anomaly detection	Explicit features, labeled evaluation data, false-positive measures and privacy review

Machine learning and blockchain are not included merely to make the project sound advanced. Any such extension must solve a defined problem and provide measured evidence. The current release demonstrates security controls whose behavior can be inspected directly.

13 Conclusion

The project implements an explainable file-upload pipeline with account access, layered validation, normalized images, encrypted storage, owner-only retrieval, optional MFA and integrity-aware activity logging. Its SDLC artifacts connect each major requirement to implementation and verification. The result supports a local academic demonstration and provides an explicit basis for further development. Public deployment and additional formats require the separate operational and security work described in this report.

References

[R1] OWASP. File Upload Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[R2] OWASP. Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[R3] Souppaya, M., Scarfone, K. and Dodson, D. NIST SP 800-218, Secure Software Development Framework Version 1.1. February 2022. https://csrc.nist.gov/pubs/sp/800/218/final

[R4] Python Cryptographic Authority. Fernet documentation. https://cryptography.io/en/stable/fernet/

[R5] PyOTP. Official documentation. https://pyauth.github.io/pyotp/

[R6] Pallets. Flask Security Considerations. https://flask.palletsprojects.com/en/stable/web-security/

[R7] Pillow. Image module reference. https://pillow.readthedocs.io/en/stable/reference/Image.html

[R8] ClamAV. Scanning documentation. https://docs.clamav.net/manual/Usage/Scanning.html

[R9] GitHub. Code scanning documentation. https://docs.github.com/en/code-security/concepts/code-scanning/code-scanning

Web references consulted during project preparation on 16-17 September 2026.